

第六章

1. 软件设计目标中，“代码按照需求要求执行”指的是？（单选题）

选项：

- A: 健壮性
- B: 正确性
- C: 高效性
- D: 可复用性

正确答案：B

解析：

- 1. **正确性**：核心定义是软件功能完全符合需求规格说明，程序输出、逻辑与需求一致，对应题干描述；
- 2. **健壮性**：系统处理非法输入、异常场景不崩溃；
- 3. **高效性**：程序时间、空间资源消耗少；
- 4. **可复用性**：代码、模块可在多个场景重复使用

2. 在计算器程序中，增加对“除数不能为 0”的判断，主要是为了提高程序的？（单选题）

选项：

- A: 正确性
- B: 高效性
- C: 健壮性
- D: 可维护性

正确答案：C

解析：健壮性（鲁棒性）指程序遇到非法输入、异常条件时，不会崩溃、报错退出，而是给出友好提示。除数为 0 属于非法运算输入，增加校验逻辑属于异常防护，提升健壮性。

正确性仅保证正常输入下结果正确，不处理异常输入；

高效、可维护和本题异常校验无关。

3. 下列哪种手段在提高代码效率的同时，可能会降低代码的可读性、扩展性和复用性？（单选题）

选项：

- A: 消除远程调用
- B: 处理嵌套循环
- C: 消除或减少函数调用
- D: 压缩存储数据

正确答案：C

解析：函数的作用是封装逻辑、拆分代码，提升可读性与复用性；为了性能取消函数调用，会把大量代码平铺在一处，代码冗长杂乱，可读性大幅下降，后期难以扩展复用。

A、B、D 都是常规性能优化手段，不会破坏代码结构。

4. 实现类的复用时，将新创建类定义为已设计好的类的子类，这种方法称为？（单选题）

选项：

- A: 聚合复用
- B: 依赖复用
- C: 继承复用
- D: 关联复用

正确答案：C

解析：继承复用：通过子类 `extends` 父类的方式复用父类属性与方法；

聚合复用：组合 / 聚合方式，将已有类作为成员变量复用；依赖、关联只是类间弱关系，不属于主流复用方案。

5. 软件设计中，将“业务逻辑”与“界面逻辑”分开的主要目的是？（单选题）

选项：

- A: 提高执行效率
- B: 降低耦合度，提高可维护性
- C: 减少内存占用
- D: 增加代码行数

正确答案：B

解析：业务和界面分离（如 MVC 分层），二者低耦合：修改页面不用改动业务代码，修改业务逻辑不用改动界面，大幅提升可维护性；分层不会提升效率、减少内存，也不会刻意增加代码行数。

6. 简单工厂模式的主要缺点是？（单选题）

选项：

- A: 无法实例化对象
- B: 客户端代码过于复杂
- C: 增加新功能时需要修改工厂类的判断逻辑，违背“开-闭”原则
- D: 不支持多态

正确答案：C

解析：开闭原则：对扩展开放，对修改关闭。简单工厂只有一个统一工厂类，新增产品时，必须修改工厂内部 if/switch 判断分支，改动原有代码，违反开闭原则。

A 错误，简单工厂核心功能就是实例化对象；B 错误，客户端只需传入标识，代码简洁；D 错误，简单工厂支持多态。

7. 工厂方法模式通过什么方式克服了简单工厂模式违背“开-闭”原则的缺点？（单选题）

选项：

- A: 取消了工厂类
- B: 使用多态性，将实例化逻辑移至具体子类工厂
- C: 减少了类的数量
- D: 禁用了继承

正确答案：B

解析：工厂方法拆分出**抽象工厂接口**和**多个具体工厂子类**；新增产品时，只新增对应的产品类 + 工厂子类，不修改原有任何代码，依靠多态实现创建，符合开闭原则。

A 错误，仍存在工厂类；C 错误，相比简单工厂，类数量更多；D 错误，大量使用继承与多态。

8. 设计模式中，主要用于描述类或对象怎样交互和分配职责的是？（单选题）

选项：

- A: 创建型模式
- B: 结构型模式

- C: 行为型模式
- D: 静态模式

正确答案: C

解析: GoF23 种设计模式三大分类:

1. 创建型: 负责对象创建(工厂、单例、建造者等);
2. 结构型: 负责类 / 对象组合、搭建结构(代理、适配器等);
3. 行为型: 负责对象交互、职责分配、算法流程(观察者、策略等);

无 “静态模式” 分类。

9. 下列不属于面向对象设计原则的是? (单选题)

选项:

- A: 单一职责原则
- B: 迪米特法则
- C: 快速原型原则
- D: 里氏代换原则

正确答案: C

解析: 经典面向对象 SOLID + 迪米特等原则包含: 单一职责、开闭、里氏替换、依赖倒置、接口隔离、迪米特法则;

快速原型是软件开发过程模型, 不是设计原则。

10. 软件逆向工程的主要目的是? (单选题)

选项:

- A: 编写新的代码
- B: 分析系统成分及关系, 提取设计信息
- C: 优化数据库查询速度
- D: 彻底删除既存系统

正确答案: B

解析: 逆向工程: 针对无源码、文档缺失的现有软件, 反推代码结构、模块关系、原始设计思路, 还原系统设计信息; A、C、D 均不是逆向工程目标。

11. 在度量面向对象软件质量时，内聚度是指？（单选题）

选项：

- A: 一个类作用于其他类的程度
- B: 一个模块内部各成分之间相互关联的程度
- C: 继承树的深度
- D: 方法数量的多少

正确答案：B

解析：内聚：衡量**模块内部**元素关联性强弱，高内聚代表一个模块只做一件事；

A 描述的是耦合度（模块之间依赖程度）；C、D 是面向对象复杂度度量指标，和内聚无关。

12. 哪个设计模式可以避免不必要的函数调用，常用于连接对远程函数的调用？（单选题）

选项：

- A: 单例模式
- B: 工厂模式
- C: 代理(Proxy)模式
- D: 观察者模式

正确答案：C

解析：代理模式：提供一个代理对象控制对真实对象的访问，远程代理专门用于封装远程调用，增加缓存、拦截逻辑，避免重复发起远程请求、减少无效函数调用；

单例控制实例唯一，工厂负责创建对象，观察者处理事件通知，均不符合题意。

13. 下面哪些是软件设计的主要目标？（多选题）

选项：

- A: 正确性
- B: 健壮性
- C: 高效性
- D: 可复用性和可维护性

正确答案：ABCD

解析：软件设计四大核心质量目标：

1. 正确性：满足需求；
2. 健壮性：容错抗异常；

2. 高效性：时间、空间性能优良；4. 可复用、可维护：便于迭代复用。

14. 为了提高程序的健壮性，需要防止哪些方面的错误？（多选题）

选项：

- A: 用户输入错误
- B: 数据通信错误
- C: 开发过程中的错误设计
- D: 开发过程中的错误实现

正确答案：ABCD

解析：健壮性覆盖全场景异常防护：

1. 外部异常：用户非法输入、网络通信中断 / 乱码；
2. 内部隐患：设计缺陷、代码实现 bug，都会导致系统异常崩溃，健壮性设计需要全部规避处理。

15. 提高执行效率（时间效率）可以从哪些方面入手？（多选题）

选项：

- A: 优化嵌套循环
- B: 消除不必要的远程调用
- C: 增加不必要的函数调用
- D: 利用后台线程预取数据

正确答案：ABD

解析：A 优化循环、B 减少远程网络开销、D 异步预加载数据，均能缩短执行耗时；

C 多余函数调用会增加栈开销，降低运行效率，排除。

16. 存储效率（空间效率）的度量通常关注？（多选题）

选项：

- A: RAM 的大小
- B: 代码本身的规模
- C: 程序员的打字速度
- D: 磁盘驱动器的大小

正确答案：ABD

解析：空间效率衡量程序占用硬件存储资源多少：运行内存 RAM、程序代码体积、持久化磁盘占用；程序员打字速度和存储资源无关。

17. 实现类复用的技术手段有？（多选题）

选项：

- A: 继承
- B: 聚合
- C: 依赖
- D: 剪贴代码

正确答案：ABC

解析：正规面向对象复用方案：

1. 继承复用：子类复用父类；
2. 聚合 / 组合复用：将已有类作为成员；
3. 依赖复用：方法入参传入外部类，调用其能力；

剪贴代码属于复制粘贴，是劣质复用，会产生冗余，不属于规范复用手段。

18. 简单工厂模式中的“运算工厂”类主要负责？（多选题）

选项：

- A: 编写加减乘除的具体算法
- B: 决定实例化哪一个具体的运算类
- C: 接收客户端的运算符号
- D: 存储所有运算结果

正确答案：BC

解析：简单工厂职责划分：

1. 工厂类：接收客户端传入标识（运算符），判断并创建对应运算子类实例（B、C）；
2. 具体运算类（加 / 减 / 乘 / 除）：实现各自计算算法（A 不属于工厂职责）；
3. 运算结果由客户端或运算子类保存，工厂不存储结果（D 错误）。

19. 工厂方法模式的优点包括？（多选题）

选项：

- A: 保持了封装对象创建过程的优点
- B: 克服了违背“开-闭”原则的缺点
- C: 降低了客户程序与产品对象的耦合
- D: 减少了类的数量，降低了开发量

正确答案：ABC

解析：A: 依然将对象创建逻辑封装在工厂，客户端不用 new 产品；

B: 新增产品只需新增工厂子类，不修改原有代码，符合开闭原则；

C: 客户端只依赖抽象工厂、抽象产品，不依赖具体实现类，耦合降低；

D 错误：相比简单工厂，工厂方法需要新增抽象工厂、多个具体工厂，类数量增多。

20. 设计模式一般包含的基本要素有？（多选题）

选项：

- A: 模式名称
- B: 问题
- C: 解决方案
- D: 效果

正确答案：ABCD

解析：标准 GoF 设计模式四要素：

1. 模式名称：统一命名方便交流；
2. 问题：描述该模式适用的业务场景；
3. 解决方案：类结构、交互代码实现方案；
4. 效果：使用该模式带来的优缺点、权衡取舍。